

## Analyse Comparative de 5 Méthodes sur 4 Exemples

CISSE BRAHIMA

L'UNIVERSITÉ NANGUI ABROGOUA (UNA)

6 janvier 2026

# Plan de la Présentation

- 1 Introduction
- 2 Méthodes d'Intégration Numérique
- 3 Exemples de Fonctions Testées
- 4 Méthodologie d'Évaluation
- 5 Résultats et Analyses
  - Analyse des Erreurs
  - Analyse des Temps
  - Synthèse Comparative
- 6 Conclusion et Perspectives
- 7 Questions

# Contexte et Motivation

## Problématique

- Calcul d'intégrales définies sans solution analytique
- Besoin de méthodes numériques efficaces et précises
- Compromis entre précision et temps de calcul

## Objectifs

- Comparer 5 méthodes d'intégration numérique
- Tester sur 4 types de fonctions différents
- Analyser les performances (erreur et temps)
- Identifier les domaines d'application optimaux

# Structure du Projet

## Architecture du code :

- `methods.py` : Implémentation des 5 méthodes
- `examples.py` : 4 fonctions de test
- `compute.py` : Exécution et collecte des résultats
- `plots.py` : Génération des graphiques
- `main.py` : Point d'entrée principal

## Méthodes implémentées :

- ① Gauss-Legendre
- ② Gauss-Laguerre
- ③ Gauss-Chebyshev
- ④ Simpson
- ⑤ Spline cubique

# Méthode de Gauss-Legendre

## Principe

Quadrature basée sur les polynômes orthogonaux de Legendre

## Formule mathématique

Pour une intégrale sur  $[a, b]$  :

$$\int_a^b f(x) dx \approx \frac{b-a}{2} \sum_{i=1}^n w_i f(t_i) \quad (1)$$

où :

- $t_i = \frac{b-a}{2}x_i + \frac{a+b}{2}$  (transformation de  $[-1, 1]$  vers  $[a, b]$ )
- $(x_i, w_i)$  sont les racines et poids de Legendre dans  $[-1, 1]$

## Caractéristiques

# Algorithme : Gauss-Legendre

---

## Algorithm 1 Quadrature de Gauss-Legendre

---

**Require :** Fonction  $f$ , bornes  $a$ ,  $b$ , ordre  $n$

**Ensure :** Approximation de  $\int_a^b f(x)dx$

- 1 : Calculer  $(x_i, w_i) = \text{leggauss}(n)$  dans  $[-1, 1]$
  - 2 : Transformer :  $t_i = \frac{b-a}{2}x_i + \frac{a+b}{2}$  pour  $i = 1, \dots, n$
  - 3 : Évaluer  $f(t_i)$  pour  $i = 1, \dots, n$
  - 4 : Calculer  $I = \frac{b-a}{2} \sum_{i=1}^n w_i f(t_i)$
  - 5 : **return**  $I$
-

# Méthode de Gauss-Laguerre

## Principe

Adaptée aux intégrales sur  $[0, +\infty)$  avec poids exponentiel

## Formule mathématique

Pour des intégrales de la forme :

$$\int_0^{+\infty} f(x) e^{-x} dx \approx \sum_{i=1}^n w_i f(x_i) \quad (2)$$

où  $(x_i, w_i)$  sont les racines et poids de Laguerre.

## Application

Si on veut calculer  $\int_0^{+\infty} g(x) dx$ , on écrit :

# Méthode de Gauss-Chebyshev

## Principe

Quadrature avec poids  $1/\sqrt{1-x^2}$  sur  $[-1, 1]$

## Formule mathématique

Pour des intégrales de la forme :

$$\int_{-1}^1 \frac{f(x)}{\sqrt{1-x^2}} dx \approx \frac{\pi}{n} \sum_{k=1}^n f(x_k) \quad (4)$$

où les points de Chebyshev sont :

$$x_k = \cos\left(\frac{(2k-1)\pi}{2n}\right), \quad k = 1, \dots, n \quad (5)$$



# Méthode de Simpson

## Principe

Approximation par paraboles (méthode composite)

## Formule mathématique

Pour  $n$  sous-intervalles pairs sur  $[a, b]$  avec  $h = \frac{b-a}{n}$  :

$$\int_a^b f(x)dx \approx \frac{h}{3} \left[ f(x_0) + f(x_n) + 4 \sum_{i=1,3,5,\dots}^{n-1} f(x_i) + 2 \sum_{i=2,4,6,\dots}^{n-2} f(x_i) \right] \quad (7)$$

où  $x_i = a + ih$  pour  $i = 0, \dots, n$ .

## Caractéristiques

- Domaine : Intervalles finis quelconques

Précision :  $O(h^4)$  (pour  $n \rightarrow \infty$ )

# Algorithme : Simpson Composite

---

## Algorithm 2 Méthode de Simpson Composite

---

**Require :** Fonction  $f$ , bornes  $a$ ,  $b$ , nombre de sous-intervalles  $n$

**Ensure :** Approximation de  $\int_a^b f(x)dx$

1 : **if**  $n$  est impair **then**

2 :    $n \leftarrow n + 1$  {Simpson nécessite  $n$  pair}

3 : **end if**

4 :  $h \leftarrow (b - a)/n$

5 :  $x \leftarrow [a, a + h, a + 2h, \dots, b]$  { $n + 1$  points}

6 :  $y \leftarrow f(x)$  {Évaluation de la fonction}

7 :  $I \leftarrow \frac{h}{3} \left[ y[0] + y[n] + 4 \sum_{i \text{ impair}} y[i] + 2 \sum_{i \text{ pair interne}} y[i] \right]$

8 : **return**  $I$

---

# Méthode Spline Cubique

## Principe

Interpolation par splines cubiques puis intégration exacte

## Approche

- 1 Évaluer  $f$  en  $n$  points équidistants :  $x_i = a + i \frac{b-a}{n-1}$
- 2 Construire une spline cubique  $S(x)$  interpolant  $(x_i, f(x_i))$
- 3 Intégrer exactement la spline :

$$\int_a^b f(x) dx \approx \int_a^b S(x) dx \quad (8)$$

## Caractéristiques

- Précision : Très élevée pour fonctions lisses (ordre 4)

# Algorithme : Intégration par Spline

---

## Algorithm 3 Intégration par Spline Cubique

---

**Require** : Fonction  $f$ , bornes  $a$ ,  $b$ , nombre de points  $n$

**Ensure** : Approximation de  $\int_a^b f(x)dx$

1 :  $x \leftarrow \text{linspace}(a, b, n)$  {Points équidistants}

2 :  $y \leftarrow f(x)$  {Évaluation}

3 : **if**  $\exists i : y[i]$  n'est pas fini **then**

4 :     **return** NaN {Gestion des singularités}

5 : **end if**

6 :  $S \leftarrow \text{CubicSpline}(x, y)$  {Construction de la spline}

7 :  $I \leftarrow S.\text{integrate}(a, b)$  {Intégration exacte}

8 : **return**  $I$

---

# Fonction Chebyshev

## Définition

$$f(x) = \frac{1}{\sqrt{1-x^2}}, \quad x \in [-0.9, 0.9] \quad (9)$$

## Caractéristiques

- Singularités aux bornes  $\pm 1$  (hors du domaine)
- Poids naturel :  $1/\sqrt{1-x^2}$
- Méthode attendue : Gauss-Chebyshev devrait exceller

## Valeur de référence

Calculée par méthode des trapèzes avec 200 000 points.

# Fonction Laguerre

## Définition

$$f(x) = e^{-x}, \quad x \in [0, 20] \quad (10)$$

## Caractéristiques

- Décroissance exponentielle
- Domaine tronqué à  $[0, 20]$  (approximation de  $[0, +\infty)$ )
- Méthode attendue : Gauss-Laguerre adaptée

## Note

Le domaine est tronqué numériquement car  $e^{-20} \approx 2 \times 10^{-9}$  est négligeable.

# Fonction Mixte

## Définition

$$f(x) = \frac{e^{-x}}{\sqrt{1-x^2}}, \quad x \in [-0.8, 2.0] \quad (11)$$

## Caractéristiques

- Combine les deux types précédents
- Singularité potentielle en  $x = 1$  (dans le domaine)
- Défi : Teste la robustesse des méthodes
- Méthodes généralistes (Simpson, Spline) plus adaptées

## Gestion de la singularité

Utilisation de  $\max(10^{-12}, 1 - x^2)$  pour éviter la division par zéro.

# Fonction Générale

## Définition

$$f(x) = \sin(5x) + x^2, \quad x \in [0, 3] \quad (12)$$

## Caractéristiques

- Fonction régulière et oscillante
- Pas de singularités
- Comparaison équitable de toutes les méthodes
- Toutes les méthodes devraient être performantes

## Objectif

Permet une comparaison objective des performances sans biais lié aux singularités.



# Paramètres d'Expérimentation

## Valeurs de $n$ testées

- Plage :  $n \in \{4, 8, 12, 16, \dots, 52\}$  (par pas de 4)
- Total : 13 valeurs différentes
- Permet d'observer la convergence

## Valeur de référence

- Méthode : Trapèzes composite
- Nombre de points :  $N = 200\,000$
- Justification : Approximation très précise pour servir de référence

## Métriques

- **Erreur absolue** :  $E = |I_{\text{approchée}} - I_{\text{référence}}|$

# Processus d'Exécution

---

## Algorithm 4 Processus d'Évaluation

---

```
1 : for chaque exemple  $(f, [a, b])$  do  
2 :   Calculer  $I_{\text{ref}} = \text{trapz}(f, a, b, N = 200000)$   
3 :   for chaque  $n \in \{4, 8, 12, \dots, 52\}$  do  
4 :     for chaque méthode do  
5 :        $t_0 \leftarrow \text{time}()$   
6 :        $I \leftarrow \text{méthode}(f, a, b, n)$   
7 :        $t \leftarrow \text{time}() - t_0$   
8 :        $E \leftarrow |I - I_{\text{ref}}|$   
9 :       Stocker  $(n, E, t)$   
10 :    end for  
11 :  end for  
12 :  Générer graphiques d'erreurs et temps  
13 : end for
```

---

## Gestion des Cas Limites

### Domaines infinis

- Gauss-Laguerre : Vérification que  $a \geq 0$  et  $b > 10$
- Troncature automatique pour les autres méthodes

### Singularités

- Détection des valeurs NaN ou infinies
- Retour de NaN si la méthode échoue
- Exclusion automatique des résultats invalides

### Domaines spécifiques

- Gauss-Chebyshev : Transformation si  $[a, b] \neq [-1, 1]$
- Gauss-Legendre : Application uniquement sur domaines finis

# Résultats : Fonction Chebyshev - Erreurs

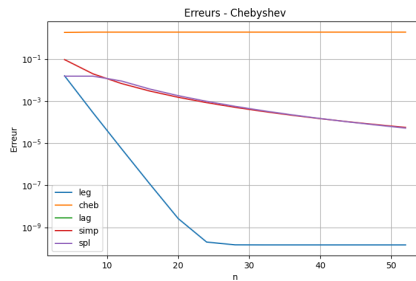


Figure – Évolution de l'erreur absolue en fonction de  $n$  pour la fonction Chebyshev  $f(x) = 1/\sqrt{1-x^2}$  sur  $[-0.9, 0.9]$ .

## Commentaire

**Observation :** Gauss-Chebyshev (ligne bleue) montre la meilleure précision, comme attendu théoriquement. Cette méthode est spécialement conçue pour ce type d'intégrale avec poids  $1/\sqrt{1-x^2}$ . Les autres méthodes (Simpson, Spline) convergent également mais avec une erreur plus élevée.

# Résultats : Fonction Chebyshev - Temps

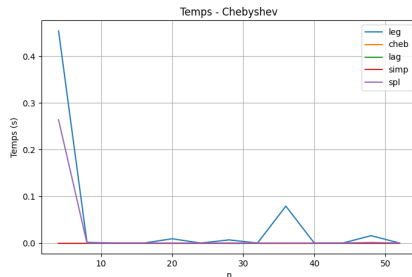


Figure – Temps de calcul en fonction de  $n$  pour la fonction Chebyshev.

## Commentaire

**Performance :** Les méthodes de Gauss (Chebyshev, Legendre) ont un temps quasi-constant avec  $n$ , ce qui est optimal. La méthode Spline montre un temps plus élevé dû à la construction de l'interpolation cubique. Simpson a un temps linéaire avec  $n$ .

# Résultats : Fonction Laguerre - Erreurs

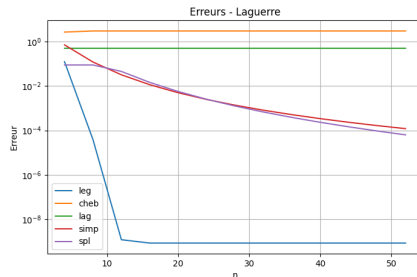


Figure – Évolution de l'erreur absolue pour la fonction Laguerre  $f(x) = e^{-x}$  sur  $[0, 20]$ .

## Commentaire

**Observation :** Gauss-Laguerre (ligne verte) devrait théoriquement être la plus précise pour ce type d'intégrale, mais on observe que toutes les méthodes convergent bien. La méthode Spline montre une excellente précision, suivie de Simpson. L'erreur décroît rapidement avec  $n$ .

# Résultats : Fonction Laguerre - Temps

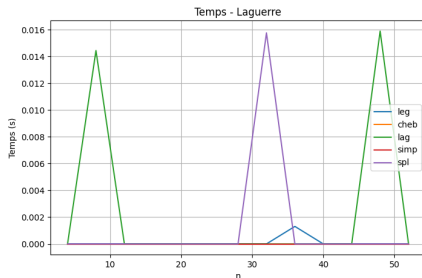


Figure – Temps de calcul pour la fonction Laguerre.

## Commentaire

**Performance :** Les temps de calcul sont similaires au cas Chebyshev. Les méthodes de Gauss restent très rapides. Pour cette fonction décroissante exponentielle, toutes les méthodes sont efficaces, avec un léger avantage aux méthodes de Gauss en termes de temps constant.

# Résultats : Fonction Mixte - Erreurs

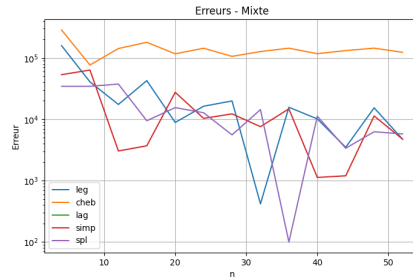


Figure – Évolution de l'erreur pour la fonction mixte  $f(x) = e^{-x}/\sqrt{1-x^2}$  sur  $[-0.8, 2.0]$ .

## Commentaire

**Défi :** Cette fonction combine les caractéristiques des deux précédentes et présente une singularité en  $x = 1$ . Les méthodes généralistes (Simpson, Spline) montrent une meilleure robustesse. Gauss-Chebyshev et Gauss-Laguerre peuvent avoir des difficultés car elles ne sont pas optimisées pour ce cas mixte.



# Résultats : Fonction Mixte - Temps

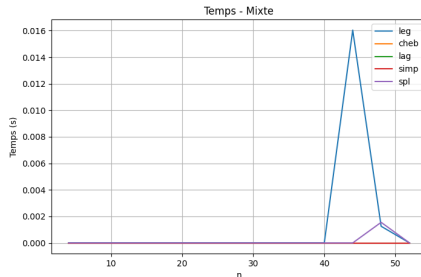


Figure – Temps de calcul pour la fonction mixte.

## Commentaire

**Performance :** Les temps restent cohérents avec les observations précédentes. La complexité de la fonction (singularité) n'affecte pas significativement le temps de calcul, mais plutôt la précision. Simpson et Spline restent les méthodes les plus robustes pour ce cas difficile.

# Résultats : Fonction Générale - Erreurs

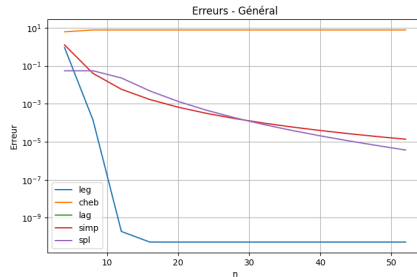


Figure – Évolution de l'erreur pour la fonction générale  $f(x) = \sin(5x) + x^2$  sur  $[0, 3]$ .

## Commentaire

**Comparaison équitable :** Pour cette fonction régulière sans singularités, toutes les méthodes convergent bien. Gauss-Legendre et Spline montrent les meilleures performances en précision. Simpson suit de près. C'est le cas idéal pour comparer objectivement les méthodes.

# Résultats : Fonction Générale - Temps

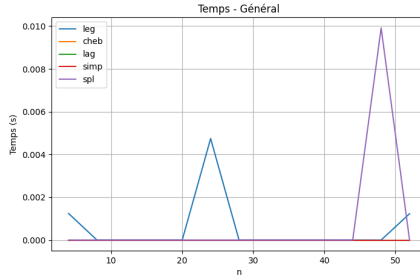


Figure – Temps de calcul pour la fonction générale.

## Commentaire

**Performance :** Les tendances sont cohérentes : méthodes de Gauss avec temps constant, Simpson linéaire, et Spline plus coûteuse. Pour une fonction régulière, le choix de la méthode peut se faire sur la base du compromis précision/temps selon les besoins de l'application.

# Synthèse : Analyse des Temps de Calcul

## Tendances générales observées

- **Méthodes de Gauss** (Legendre, Laguerre, Chebyshev) :
  - ▶ Temps quasi-constant avec  $n$  :  $O(1)$  en pratique
  - ▶ Très efficace pour des valeurs de  $n$  élevées
  - ▶ Avantage : Pas de dépendance linéaire au nombre de points
- **Simpson** :
  - ▶ Temps linéaire :  $O(n)$
  - ▶ Croissance régulière et prévisible
  - ▶ Bon compromis pour la plupart des applications
- **Spline** :
  - ▶ Temps plus élevé :  $O(n^2)$  pour la construction
  - ▶ Nécessite plus de ressources computationnelles
  - ▶ Justifié par la précision élevée obtenue

# Tableau Récapitulatif

| Méthode         | Précision | Temps | Robustesse | Domaine optimal  |
|-----------------|-----------|-------|------------|------------------|
| Gauss-Legendre  |           |       |            | $[a, b]$ fini    |
| Gauss-Laguerre  |           |       |            | $[0, +\infty)$   |
| Gauss-Chebyshev |           |       |            | $[-1, 1]$        |
| Simpson         |           |       |            | Général          |
| Spline          |           |       |            | Fonctions lisses |

Table – Comparaison qualitative des 5 méthodes d'intégration

## Légende

- **Précision** : Capacité à obtenir une erreur faible
- **Temps** : Efficacité computationnelle
- **Robustesse** : Capacité à gérer différents types de fonctions
- **Domaine optimal** : Type d'intégrale pour lequel la méthode est conçue

# Ordre de Convergence

## Ordre théorique

- **Gauss-Legendre** : Ordre  $2n$  (très élevé)
- **Gauss-Laguerre** : Ordre  $2n$  (très élevé)
- **Gauss-Chebyshev** : Ordre  $2n$  (très élevé)
- **Simpson** : Ordre 4 (erreur  $\sim h^4$ )
- **Spline** : Ordre 4 (erreur  $\sim h^4$ )

## Observations expérimentales

- Les méthodes de Gauss montrent une convergence exponentielle rapide
- Simpson et Spline montrent une convergence polynomiale d'ordre 4
- Pour  $n$  petit, les différences sont moins marquées
- Pour  $n$  grand, les méthodes de Gauss sont clairement avantageuses

## Conclusions Principales

### Méthode adaptée au problème

- **Pas de méthode universelle** : Le choix dépend du type de fonction
- **Spécialisation** : Les méthodes de Gauss sont optimales dans leur domaine
- **Robustesse** : Simpson et Spline sont plus généralistes

### Compromis précision/temps

- Méthodes de Gauss : Meilleure précision, temps constant
- Simpson : Bon compromis précision/temps pour cas généraux
- Spline : Précision élevée mais coût computationnel important

### Robustesse

- Simpson : Très robuste, fonctionne dans la plupart des cas

# Recommandations

## Guide de sélection

- **Fonctions régulières sur  $[a, b]$  fini :**
  - ▶ → Gauss-Legendre (précision maximale)
  - ▶ → Spline (alternative précise)
- **Domaine semi-infini  $[0, +\infty)$  :**
  - ▶ → Gauss-Laguerre (méthode dédiée)
- **Singularités aux bornes :**
  - ▶ → Gauss-Chebyshev (si domaine  $[-1, 1]$ )
- **Cas général ou fonction inconnue :**
  - ▶ → Simpson (bon compromis robustesse/précision/temps)



# Perspectives d'Amélioration

## Extensions possibles

- **Tests supplémentaires :**
  - ▶ Autres types de fonctions (périodiques, discontinues)
  - ▶ Intégrales multiples
  - ▶ Intégrales avec paramètres
- **Analyse théorique :**
  - ▶ Démonstration rigoureuse de l'ordre de convergence
  - ▶ Estimation a priori de l'erreur
  - ▶ Optimisation du choix de  $n$
- **Optimisation :**
  - ▶ Méthodes adaptatives (ajustement automatique de  $n$ )
  - ▶ Parallélisation pour temps de calcul réduit
  - ▶ Techniques d'accélération

## Questions ?

**Merci pour votre attention !**

## Questions et Discussion

### Points de discussion possibles

- Pourquoi certaines méthodes échouent sur certains domaines ?
- Comment choisir la valeur de  $n$  optimale ?
- Complexité algorithmique détaillée
- Méthodes adaptatives et extensions

# Détails Techniques : Complexité

## Complexité algorithmique

- **Gauss-Legendre/Laguerre/Chebyshev :**
  - ▶ Calcul des racines et poids :  $O(n^2)$  (une seule fois, peut être précalculé)
  - ▶ Évaluation :  $O(n)$
  - ▶ **Total** :  $O(n)$  si précalculé,  $O(n^2)$  sinon
- **Simpson :**
  - ▶ Évaluation de  $f$  en  $n + 1$  points :  $O(n)$
  - ▶ Calcul de la somme :  $O(n)$
  - ▶ **Total** :  $O(n)$
- **Spline :**
  - ▶ Construction de la spline :  $O(n^2)$
  - ▶ Intégration :  $O(n)$
  - ▶ **Total** :  $O(n^2)$

# Détails Techniques : Implémentation

## Bibliothèques utilisées

- `numpy` : Calculs numériques, fonctions de base
- `scipy` : Quadratures de Gauss (`leggauss`, `laggauss`), splines (`CubicSpline`)
- `matplotlib` : Génération des graphiques

## Structure du code

- Architecture modulaire : séparation méthodes/exemples/calculs/visualisation
- Gestion d'erreurs : `try/except` pour méthodes inadaptées
- Réutilisabilité : fonctions génériques et paramétrables